

Python Internals & Foundations

A consolidated reference covering the interpreter, memory model, core data types, strings, collections, iteration, and functions — rebuilt from hands-on REPL exploration.

Author

Saket Sharma

Environment

Python 3.13 · VS Code · Windows

PYTHON_CORE / 01-BASICS

Table of Contents

Eleven chapters, rebuilt and reorganized from the original working notes — nothing omitted, everything de-duplicated.

01	How Python Runs — Bytecode & the Python Virtual Machine	p. 3
02	The Shell, Modules & Early Errors	p. 6
03	Mutability, References & Memory	p. 8
04	Numbers — Arithmetic, Precision & Bases	p. 11
05	Booleans, Sets & Comparisons	p. 16
06	Strings — Slicing, Methods & Formatting	p. 17
07	Lists — Indexing, Slicing & Comprehensions	p. 21
08	Dictionaries & Tuples	p. 24
09	Files, Iterators & Loop Internals	p. 28
10	Functions — Definitions, Arguments & Scope	p. 33
11	Closing Summary & What Comes Next	p. 37

► HOW THIS NOTEBOOK IS ORGANIZED

The original notes mixed live REPL transcripts with bilingual (Hindi–English) reasoning written in the moment. This edition keeps every concept and every code example, removes repeated/duplicate REPL blocks, and reframes the reasoning in clear English — with the original Hinglish phrasing preserved in gold wherever it captured something worth keeping in its own words.

CHAPTER 01

How Python Runs — Bytecode & the Python Virtual Machine

Before writing a single line of logic, it helps to know what actually happens between typing `python script.py` and seeing output on screen.

■ From script to bytecode

A `.py` script is not run directly against the hardware. Python first compiles it down to an intermediate, low-level, platform-independent form called **bytecode**. The word "compile" here is a technical convenience — it does not mean Python behaves like a traditional compiled language such as C; it simply describes this one internal translation step.

► WHY BYTECODE EXISTS

Bytecode runs faster than re-parsing raw source on every execution, and because it is platform-independent, the same bytecode can run on any machine with a compatible Python Virtual Machine.

.pyc files and __pycache__

Compiled bytecode is cached on disk as a `.pyc` file (sometimes called a "frozen binary") inside a special `__pycache__` folder. This folder is created and maintained automatically by Python — it tracks which cached bytecode corresponds to which source version.

NAMING PATTERN

`hello_world.cpython-313.pyc` — the number reflects the Python version used to compile it (here, 3.13).

DOUBLE UNDERSCORES

`__pycache__` — the double-underscore convention is used throughout Python for internals, both in folder/file names and in special methods.

► IMPORTANT LIMITATION

A `.pyc` file is only generated for **imported** modules, never for the top-level script you run directly. If a file is not imported by anything — i.e. it's a standalone, single-structure script — Python has no reason to cache its bytecode.

Whenever the source changes, Python uses an internal **diffing algorithm** (a difference-finding algorithm) to detect that the cached bytecode is stale and needs to be regenerated for the new version.

■ The Python Virtual Machine (PVM)

The PVM is a small piece of software running a continuous loop: whatever code is fed into it as data, it executes immediately, one instruction at a time. This is exactly why Python is described as an **interpreted language** — the PVM interprets and executes line by line rather than running a fully pre-compiled binary.

- The PVM does not strictly require bytecode as input — raw Python scripts can be fed into it directly, which is why a `.pyc` file isn't always generated.
- The PVM is also called the **Run-Time Engine** or the **Python interpreter**.
- Every language needs an engine like this to actually execute code — Node.js, for instance, is essentially a wrapper around its own JavaScript engine.

► BYTECODE IS NOT MACHINE CODE

Machine code is the literal, direct instruction set a CPU executes — assembly-level instructions that interact with hardware directly. Bytecode is a Python-specific intermediate representation; the PVM still has to interpret it. They are not the same thing, even though both are described as "low-level."

Python implementations

CPython is the standard, reference implementation of Python, but it isn't the only one:

IMPLEMENTATION	NOTE
CPython	The standard, default implementation — written in C.
Jython	Python implemented on top of the Java Virtual Machine.
IronPython	Python targeting the .NET runtime.
Stackless Python	A CPython variant built around microthreads, avoiding the C call stack.
PyPy	A JIT-compiled implementation, often significantly faster than CPython.

■ The pipeline, end to end

CONCEPTUAL FLOW

```
Python Script (.py)
  |
  ▼ compiled (loosely speaking) to bytecode
Bytecode (.pyc, cached under __pycache__ – imported modules only)
  |
  ▼ fed line-by-line into a continuous execution loop
Python Virtual Machine (a.k.a. Run-Time Engine / Interpreter)
  |
  ▼
Program output
```

This is the foundation for everything that follows: every quirk in this notebook — from why strings behave differently from lists under assignment, to why `is` and `==` diverge — ultimately traces back to the PVM treating every single value as an object living somewhere in memory.

Continue to Chapter 02 — using the interactive shell, importing modules, and the first errors you'll meet as a beginner.

CHAPTER 02

The Shell, Modules & Early Errors

■ Launching and using the shell

Typing `python` in a terminal launches Python's interactive shell directly. The same can be reached by right-clicking a folder to open an integrated terminal inside it. Inside the shell, pressing Enter evaluates each line immediately and prints the result on the very next line — even something as simple as `3 + 3` returns an instant answer.

▸ NOT ACTUALLY A CALCULATOR

This instant evaluation is why people informally call the shell "Python's built-in calculator" — but that's a misconception. It isn't a calculator; it's simply the REPL (Read–Eval–Print Loop) evaluating each expression and echoing its result. The shell exists for quickly testing snippets and checking outputs.

SESSION — STARTUP

PPP

```
PS C:\Users\hp\OneDrive\Desktop\Python Core\01-Basics> python
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 3 + 3
6
```

■ Modules & packages

Modules and packages are simply prewritten code that only needs to be imported to be used.

OS, SYS

PPP

```
>>> import os
>>> os.getcwd()      # returns current working directory

>>> import sys
>>> sys.platform
'win32'
```

A custom module in action

Importing a self-written module (`hello_world.py`) and calling its contents:

IMPORTING HELLO_WORLD

```
>>> import hello_world
Saket Sharma
Hello Saket
1
Hello Saket
100
Chai is good for health

>>> hello_world.chai("Ginger Tea")
Ginger Tea

>>> hello_world.chai_one
AttributeError: module 'hello_world' has no attribute 'chai_one'
```

■ Early errors worth recognizing

These are some of the first error types a beginner runs into, and each one tells you something specific:

ERROR	WHAT IT MEANS
<code>NameError</code>	name 'tea' is not defined — you referenced a variable that was never created.
<code>IndentationError</code>	expected an indented block after a statement like <code>for</code> — the block body is missing or mis-indented.
<code>KeyError</code>	'comic': no such key in the dictionary being accessed.
<code>AttributeError</code>	the object exists but has no attribute/method by that name.

A trailing `...` in the shell means Python is still waiting for more input — typically inside a multi-line block.

CHAPTER 03

Mutability, References & Memory

This is one of the most important mental models in Python: **mutability and immutability are about memory and references — not about variables and constants**. Every single thing in Python is treated as an object, and a variable is just a name pointing at one.

MUTABLE

Can be changed in place after creation — e.g. lists, dictionaries, sets.

IMMUTABLE

Cannot be changed in place once created — e.g. numbers, strings, tuples.

Python also runs **automatic garbage collection**: any object no longer referenced by any name is cleaned up and erased from memory.

■ Immutable example — numbers

When a variable holding an immutable value is reassigned, it simply points to a brand-new object; the original is left untouched (and the other variable still points at the old one).

NUMBERS ARE IMMUTABLE

```
>>> x = 10
>>> y = x
>>> x
10
>>> y
10
>>> x = 15
>>> x
15
>>> y
10 # y still points at the old object – unaffected by x's reassignment
```

■ Mutable example — lists

Lists behave completely differently. A plain assignment like `x2 = x1` does not copy the list — it makes `x2` point to the *same* object in memory as `x1`. Changing the contents through one name changes what the other name sees too.

SHARED REFERENCE

```
>>> X1 = [1, 2, 3, 4, 5, 6]
>>> X2 = X1
>>> X1[0] = 44
>>> X1
[44, 2, 3, 4, 5, 6]
>>> X2
[44, 2, 3, 4, 5, 6]  # changed too – same underlying object
```

But re-assigning the whole name to a brand new literal list breaks that shared link:

RE-ASSIGNMENT BREAKS THE LINK

```
>>> p1 = [1, 2, 3]
>>> p2 = p1
>>> p2 = [1, 2, 3]  # p2 now points to a NEW list
>>> p1[0] = 55
>>> p1
[55, 2, 3]
>>> p2
[1, 2, 3]  # untouched – different object now
```

■ Making real copies

To genuinely copy a list rather than share a reference, slicing or the dedicated copy functions are used:

COPYING WITH SLICING & COPY MODULE

```
>>> h1 = [1, 2, 3]
>>> h2 = h1[:]  # slicing creates a real copy, not a reference
>>> h1[0] = 55
>>> h1
[55, 2, 3]
>>> h2
[1, 2, 3]  # unaffected

>>> import copy
>>> h2 = copy.copy(h1)  # shallow copy
>>> h2 = copy.deepcopy(h1)  # deep copy – needed for nested/multilevel lists
```

► SLICING AS A COPY MECHANISM

`list[:]` or `list[start:end]` creates a new list object rather than referencing the original. Plain assignment (`l2 = l1`) shares the same reference; slicing gives `l2` a reference to a fresh copy of `l1` instead.

 `==` vs `is`

OPERATOR	CHECKS
<code>==</code>	Whether two objects have equal values . Returns <code>True</code> if the values are the same, regardless of memory location.
<code>is</code>	Whether two names point to the same memory reference (the same object), not just equal-looking values.

Python also has the notion of **reference counting**: an internal counter (`ref_count`) tracks how many names point to a given memory location. A note from experimentation — calling `sys.getrefcount(value)` can return a surprisingly large number for some values, partly because Python's own internal loops and caching also hold references.

► NUMBERS AND STRINGS ARE CACHED DIFFERENTLY

Python doesn't garbage-collect small numbers and short strings as eagerly as other objects, since they are accessed extremely frequently — this is an internal optimization (interning), not a rule that applies to every object.

CHAPTER 04

Numbers — Arithmetic, Precision & Bases

■ Basic arithmetic & precedence

OPERATORS

```
>>> x, y, z = 2, 3, 4
>>> x + y
5
>>> 2 ** 3
8
>>> x + y * z      # multiplication before addition
14
>>> (x + y) * z    # parentheses override precedence
20
>>> 40 + 2.23
42.23
>>> y % 2          # modulo
1
>>> z ** y
64
```

► MIXED-TYPE ERRORS

Strings and integers cannot be concatenated directly with `+`:

```
>>> 'hitesh' + 3
TypeError: can only concatenate str (not "int") to str
```

Explicit conversion functions fix this:

TYPE CONVERSION

```
>>> int(2.23)
2
>>> float(40)
40.0
>>> 'chai' + 'aurcode'    # strings DO concatenate with strings
'chaiaurcode'
```

Tuples from bare expressions

IMPLICIT TUPLES



```
>>> x, y, z
(2, 3, 4)
>>> x + 1, y * 2
(3, 6)
```

■ repr() vs str() vs print()

Three different ways of presenting a value, each with a distinct purpose:

REPRESENTATION VS DISPLAY



```
>>> repr('chai')
"'chai'"      # unambiguous, includes quotes – for debugging/dev
>>> str('chai')
'chai'        # human-readable string form
>>> print('chai')
chai          # prints the raw text, no quotes
```

■ Chained comparisons

COMPARISON CHAINS



```
>>> x < y < z
True
>>> x < y and y < z    # equivalent, written explicitly
True
>>> 33 + (1 < 2)        # True behaves as 1 in arithmetic
34
>>> 1 == 2 < 3          # chained: (1==2) and (2<3) → False
False
>>> True == 2 < 3
False
```


Fixing precision — Decimal & Fraction

EXACT ARITHMETIC



```
>>> from decimal import Decimal
>>> Decimal('0.1') + Decimal('0.1') + Decimal('0.1')
Decimal('0.3')    # exact

>>> from fractions import Fraction
>>> Fraction(2, 7)
Fraction(2, 7)
```

■ Number bases — octal, hex, binary

PREFIX	BASE	EXAMPLE	RESULT
0o	Octal	0o20	16
0x	Hexadecimal	0xFF	255
0b	Binary	0b1000	8

CONVERTING BETWEEN BASES



```
>>> oct(64)
'0o100'
>>> hex(64)
'0x40'
>>> bin(64)
'0b1000000'

>>> int('64', 8)    # interpret '64' as base-8
52
>>> int('64', 16)   # interpret '64' as base-16
100
>>> int('100000', 2) # interpret as base-2
32
>>> int('64', 2)
ValueError: invalid literal for int() with base 2: '64'    # '6' isn't a valid binary digit
```

Bit shifting

LEFT / RIGHT SHIFT



```
>>> x = 1
>>> x << 2    # left shift – multiplies by 2 per shift
4
```

■ Complex numbers

COMPLEX LITERALS



```
>>> 2 + 1j
(2+1j)
>>> (2 + 1j) * 3
(6+3j)
```

■ The random module

RANDOM



```
>>> import random
>>> random.random()           # float in [0.0, 1.0)
0.34418095069592436
>>> random.randint(1, 10)     # integer in [1, 10]
7

>>> l1 = ['lemon tea', 'masala', 'ginger tea', 'mint']
>>> random.choice(l1)          # pick one element at random
'mint'
>>> random.shuffle(l1)         # shuffles the list in place
>>> l1
['masala', 'lemon tea', 'mint', 'ginger tea']
```

CHAPTER 05

Booleans, Sets & Comparisons

■ Sets vs dictionaries

Both use curly braces, but they answer different questions: a **dictionary** stores key–value pairs, while a **set** stores single, unique values.

SET OPERATIONS

```
>>> setOne = {1, 2, 3, 4}
>>> setOne & {1, 3}      # intersection
{1, 3}
>>> setOne | {1, 3, 7}   # union
{1, 2, 3, 4, 7}
>>> setOne - {1, 2, 3, 4} # difference → empty set
set()
```

► WHY EMPTY SETS PRINT AS SET()

`{}` on its own is already taken — it denotes an empty dictionary (`type({})` is `dict`). So an empty set has to be written and displayed as `set()` instead.

■ Booleans are a subtype of int

BOOL INTERNALS

```
>>> type(True)
<class 'bool'>
>>> True == 1
True
>>> False == 1
False
>>> True is 1
False # equal in value, but different objects/references
>>> True + 4
5
```

`True` and `1` compare equal in value but are different objects — a direct, concrete illustration of the `==` vs `is` distinction from Chapter 03.

CHAPTER 06

Strings — Slicing, Methods & Formatting

■ Indexing & slicing

BASIC INDEXING

```
>>> chai = "lemon Chai"
>>> chai[0]
'l'
>>> chai[0:6]
'lemon '
>>> chai[-1]           # negative index counts from the end
'i'
>>> chai[-1:-5]        # backward slice direction with no step → empty
''
```

Step slicing

[START:END:STEP]

```
>>> num_list = "0123456789"
>>> num_list[:]
'0123456789'
>>> num_list[3:]
'3456789'
>>> num_list[:7]
'0123456'
>>> num_list[0:7:2]     # every 2nd character
'0246'
>>> num_list[0:7:3]     # every 3rd character
'036'
```

■ Case & whitespace methods

```
.LOWER() .UPPER() .STRIP()

>>> chai = "masala chai"
>>> chai.upper()
'MASALA CHAI'
>>> chai
'masala chai' # original unchanged – strings are immutable

>>> chai = "  Masala Chai  "
>>> chai.strip() # removes leading/trailing whitespace
'Masala Chai'
```

■ Replace, split & join

```
.REPLACE() .SPLIT() .JOIN()

>>> chai = "Lemon Tea"
>>> chai.replace("Lemon", "Ginger")
'Ginger Tea'

>>> chai = "Lemon, Ginger, Masala, Mint"
>>> chai.split() # splits on whitespace by default
['Lemon,', 'Ginger,', 'Masala,', 'Mint']
>>> chai.split(", ") # split on a custom delimiter
['Lemon', 'Ginger', 'Masala', 'Mint']

>>> chai_variety = ["Lemon", "Masala", "Ginger"]
>>> "-".join(chai_variety)
'Lemon-Masala-Ginger'
>>> ", ".join(chai_variety)
'Lemon, Masala, Ginger'
```

■ Searching inside strings

```
.FIND() .COUNT() IN

>>> chai = "Masala Chai"
>>> chai.find("Chai")
7
>>> chai.find("chai")      # case-sensitive – not found
-1

>>> chai = "Masala Chai Chai Chai"
>>> chai.count("Chai")
3

>>> "Masala" in "Masala Chai"
True
```

`.find()` returns `-1` when the substring isn't present, rather than raising an error.

■ String formatting, length & iteration

```
.FORMAT() LEN() FOR-IN

>>> order = "I ordered {} cups of {} chai"
>>> order.format(2, "Masala")  # {} are placeholders, filled in order
'I ordered 2 cups of Masala chai'

>>> len("Masala Chai")
11

>>> for letter in "Chai":
...     print(letter)
C
h
a
i
```

■ Escape characters & raw strings

\N \" RAW STRINGS

```
>>> chai = 'He Said, \"Masala Chai is Awesome\" '
>>> chai
'He Said, "Masala Chai is Awesome" '

>>> chai = "Masala\nChai"
>>> print(chai)
Masala
Chai

>>> chai = r"Masala\nChai"    # r = raw string, escapes are literal text
>>> print(chai)
Masala\nChai
```

► RAW STRINGS STILL CAN'T END IN AN UNESCAPED BACKSLASH

A raw string like `r"c:\user\pwd\"` raises a `SyntaxError` ("unterminated string literal") — the trailing backslash still tries to escape the closing quote, even in raw mode. Doubling the backslashes (`r"c:\user\\pwd\"`) avoids the problem.

REGULAR VS RAW STRINGS, PATHS

```
>>> chai = "c:\user\pwd"
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes...

>>> chai = "c:\\user\\pwd"    # escape each backslash manually
>>> print(chai)
c:\user\pwd

>>> chai = r"c:\user\pwd"    # or just use a raw string
>>> print(chai)
c:\user\pwd
```

► STRINGS ARE IMMUTABLE, BUT THEY SUPPORT RICH OPERATIONS

Every method shown above (`.upper()`, `.replace()`, `.strip()`, etc.) returns a *new* string rather than modifying the original in place — consistent with the immutability discussed in Chapter 03.

CHAPTER 07

Lists — Indexing, Slicing & Comprehensions

■ Indexing & slicing

BASIC LIST SLICING

```
>>> tea = ["Black", "Green", "oolong", "White"]
>>> tea[0]
'Black'
>>> tea[-1]
'White'
>>> tea[:2]
['Black', 'Green']
>>> tea[-1:0]      # backward direction with no step → empty
[]
```

■ Slice assignment

Assigning to a slice replaces that range — and the replacement doesn't need to be the same length as what it replaces:

SLICE ASSIGNMENT

```
>>> tea = ["Black", "Green", "oolong", "White"]
>>> tea[1:2] = "Lemon"      # a STRING is iterable – explodes into chars!
>>> tea
['Black', 'L', 'e', 'm', 'o', 'n', 'oolong', 'White']
```

► ALWAYS WRAP THE REPLACEMENT IN A LIST

Assigning a bare string to a slice iterates the string character-by-character, splitting it across multiple list elements. Wrapping it as `["Lemon"]` inserts it as a single element instead:

CORRECT SLICE ASSIGNMENT

```
>>> tea = ["Black", "Green", "oolong", "White"]
>>> tea[1:2] = ["Lemon"]
>>> tea
['Black', 'Lemon', 'oolong', 'White']

>>> tea[1:1] = ["test", "test"]  # inserting into an empty slice = insertion at that
point
>>> tea
['Black', 'test', 'test', 'Lemon', 'oolong', 'White']

>>> tea[1:3] = []                # assigning empty list = deletion
>>> tea
['Black', 'Lemon', 'oolong', 'White']
```

■ Iterating, membership & common methods

FOR-IN, APPEND, POP, REMOVE, INSERT

```
>>> tea = ["Black", "Green", "oolong", "White"]
>>> for t in tea:
...     print(t, end="-")
Black-Green-oolong-White-

>>> tea.append("masala")
>>> "masala" in tea
True
>>> tea.pop()                # removes & returns the LAST item
'masala'

>>> tea.remove("Green")  # removes by VALUE, returns nothing
>>> tea
['Black', 'oolong', 'White']
>>> tea.insert(1, "Green")
>>> tea
['Black', 'Green', 'oolong', 'White']
```

`.pop()` echoes back the removed item; `.remove()` does not — it just removes the first matching value silently.

■ Copying lists correctly

`.COPY()`

```
>>> tea_copy = tea.copy()
>>> tea_copy.append("Lemon")
>>> tea_copy
['Black', 'Green', 'oolong', 'White', 'Lemon']
>>> tea          # original is untouched
['Black', 'Green', 'oolong', 'White']
```

Skipping `.copy()` and assigning directly would leave both names pointing at the same underlying list in memory, exactly as covered in Chapter 03.

■ List comprehensions

`[EXPR FOR X IN RANGE(...)]`

```
>>> squared_num = [x ** 2 for x in range(10)]
>>> squared_num
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

>>> cube_nums = [y ** 3 for y in range(10)]
>>> cube_nums
[0, 1, 8, 27, 64, 125, 216, 343, 512, 729]
```

CHAPTER 08

Dictionaries & Tuples

■ Creating & accessing dictionaries

BASIC DICT ACCESS

```
>>> chai_types = {"Masala": "Spicy", "Ginger": "Zesty", "Green": "Mild"}
>>> chai_types["Masala"]
'Spicy'
>>> chai_types['Green']
'Mild'
```

.get() vs direct indexing

SAFE LOOKUPS

```
>>> chai_types.get("Ginger")
'Zesty'
>>> chai_types.get("Gingery")      # missing key → None, no error

>>> chai_types["Gingery"]          # missing key → raises KeyError
KeyError: 'Gingery'
```

► PICKING THE RIGHT LOOKUP

Use `.get()` when a missing key is an expected, non-fatal case — it returns `None` (or a supplied default) instead of crashing. Use direct indexing (`d[key]`) when a missing key really should be an error.

■ Updating, iterating & deleting

UPDATE, .ITEMS(), DEL

```
>>> chai_types["Green"] = "Fresh"      # update existing key
>>> chai_types["Earl Grey"] = "Citrus" # add new key

>>> for key, value in chai_types.items():
...     print(key, value)
Masala Spicy
Ginger Zesty
Green Fresh
Earl Grey Citrus

>>> chai_types.pop("Ginger")           # remove by key, returns its value
'Zesty'
>>> chai_types.popitem()               # remove & return the LAST inserted pair
('Earl Grey', 'Citrus')
>>> del chai_types["Green"]           # delete by key directly
```

Plain iteration over a dict (`for k in d`) yields keys only, not values — `.items()` is what's needed for both at once.

■ Nested dictionaries

DICTIONARY OF DICTIONARIES

```
>>> tea_shop = {
...     "Chai": {"masala": "Spicy", "ginger": "Zesty"},
...     "Tea": {"green": "mild", "black": "strong"}
... }
>>> tea_shop["Chai"]["ginger"]
'Zesty'
```

■ Dictionary comprehensions & fromkeys

{K: V FOR ...} DICT.FROMKEYS

```
>>> squared_num = {x: x**2 for x in range(10)}
>>> squared_num
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81}
>>> squared_num.clear()
>>> squared_num
{}

>>> keys = ["Masala", "Ginger", "Lemon"]
>>> dict.fromkeys(keys, "Delicious")    # same default value for every key
{'Masala': 'Delicious', 'Ginger': 'Delicious', 'Lemon': 'Delicious'}

>>> dict.fromkeys(keys, keys)           # careful – same LIST object shared by every key
{'Masala': ['Masala', 'Ginger', 'Lemon'],
 'Ginger': ['Masala', 'Ginger', 'Lemon'],
 'Lemon':  ['Masala', 'Ginger', 'Lemon']}
```

► SHARED MUTABLE DEFAULTS

When the default value passed to `dict.fromkeys()` is mutable (like a list), every key ends up referencing the *exact same* object — mutating it through one key affects all of them. This is the same reference-sharing trap from Chapter 03, now showing up inside a dictionary.

■ Tuples — the immutable list

Why use a tuple if lists already exist? A list is mutable — once created, items can be appended, removed, or changed freely. A tuple is a similarly structured data type that stays **immutable**: once built, its contents cannot be manipulated in memory the way a list's can.

CREATING & INDEXING TUPLES

```
>>> tea_types = ("Black", "Green", "Oolong")
>>> tea_types[0]
'Black'
>>> tea_types[-1]
'Oolong'
>>> tea_types[1:]
('Green', 'Oolong')

>>> tea_types[0] = "Lemon"    # attempted mutation
TypeError: 'tuple' object does not support item assignment
```

Tuples cannot be changed directly — but the *name* referencing one can always be reassigned to point at a different tuple altogether.

■ Concatenation, membership & counting

+ IN .COUNT()

```
>>> tea_types = ("Black", "Green", "Oolong")
>>> len(tea_types)
3
>>> more_tea = ("Herbal", "Earl Grey")
>>> all_tea = tea_types + more_tea      # + creates a NEW combined tuple
>>> all_tea
('Black', 'Green', 'Oolong', 'Herbal', 'Earl Grey')

>>> "Green" in all_tea
True

>>> more_tea = ("Herbal", "Earl Grey", "Herbal")
>>> more_tea.count("Herbal")
2
```

■ Unpacking & type checking

TUPLE UNPACKING

```
>>> (black, green, oolong) = tea_types      # unwraps each element into a name
>>> black
'Black'
>>> type(tea_types)
<class 'tuple'>
```

Tuples can also be nested inside one another, the same way lists can.

CHAPTER 09

Files, Iterators & Loop Internals

■ Opening & reading files

OPEN() AND .READLINE()

```
>>> open('chai.py')
<_io.TextIOWrapper name='chai.py' mode='r' encoding='cp1252'>

>>> f = open('chai.py')
>>> f.readline()
'import time\n'
>>> f.readline()
'print("Your Chai is Here")\n'
>>> f.readline()
'username = "Saket"\n'
>>> f.readline()
'print(username)'
>>> f.readline()      # at end of file → empty string
''
```

An empty string from `.readline()` is the signal that the file is fully read — not an error, just an end-of-iteration marker.

■ The `__next__()` protocol

MANUAL ITERATION WITH `__NEXT__()`

```
>>> f = open('chai.py')
>>> f.__next__()
'import time\n'
>>> f.__next__()
'print("Your Chai is Here")\n'

# ... continues line by line until the file is exhausted ...

>>> f.__next__()
StopIteration # raised once nothing is left – the iterator's "I'm done" signal
```

The same content can be read more idiomatically with a loop:

LOOPING OVER A FILE

```
>>> for line in open('chai.py').readlines():
...     print(line)
import time

print("Your Chai is Here")
username = "Saket"
print(username)

>>> for line in open('chai.py'):      # a file object is directly iterable
...     print(line, end=' ')
```

■ while loops & truthy/falsy strings

WHILE TRUE / BREAK

```
>>> f = open('chai.py')
>>> while True:
...     line = f.readline()
...     if not line: break
...     print(line, end=' ')
```

► A REAL SYNTAX TRAP FROM THE SESSION

Writing `if not line: break` on its own continuation line without proper indentation raises `IndentationError: expected an indented block after 'while' statement`. Every line inside a block — even a one-line `if` — needs to sit at the correct, consistent indentation level.

An empty string is "falsy" — `not ""` evaluates to `True` — but a string containing only a space is non-empty and therefore "truthy":

TRUTHINESS OF STRINGS

```
>>> test = "Saket"
>>> if not test: print("chai")      # non-empty → condition is False, nothing prints
>>> test = " "
>>> if not test: print("chai")      # a single space is still non-empty → nothing prints
>>> test = ""
>>> if not test: print("chai")
chai                                # truly empty → condition is True
```

■ iter() and the iterator protocol

ITER() / NEXT()

```
>>> myList = [1, 2, 3, 4]
>>> I = iter(myList)
>>> I.__next__()
1
>>> I.__next__()
2
# ... 3, then 4 ...
>>> I.__next__()
StopIteration
```

► FILES ARE THEIR OWN ITERATOR – LISTS ARE NOT

For an open file `f`, `iter(f) is f` evaluates to `True` — referencing a file already gives you an iterable object. For a list, `iter(myNewList) is myNewList` is `False` — a list is only a reference to the underlying data; calling `iter()` on it produces a separate iterator object.

■ Iterating dictionaries & ranges

DICT_KEYITERATOR & RANGE

```
>>> D = {'a': 1, 'b': 2}
>>> for key in D.keys():
...     print(key)
a
b

>>> I = iter(D)
>>> next(I)
'a'
>>> next(I)
'b'
>>> next(I)
StopIteration
```

RANGE() IS ALSO ITERABLE, NOT A LIST

```
>>> R = range(5)
>>> R
range(0, 5)      # range objects don't expand into a list eagerly
>>> I = iter(R)
>>> next(I)
0
>>> next(I)
1
# ... 2, 3, then 4 ...
>>> next(I)
StopIteration
```

► THE PATTERN UNDERNEATH EVERY FOR-LOOP

Every `for ... in ...` loop in Python — over a list, a dict, a file, or a `range()` — is really calling `iter()` on the object once to get an iterator, then calling `next()` on it repeatedly until a `StopIteration` is raised, which the loop catches silently to end.

CHAPTER 10

Functions — Definitions, Arguments & Scope

This is exactly where the original working notes left off — only the heading for this topic existed. The content below picks up the thread directly: functions are the next logical step after the PVM, mutability, and the core built-in types, because every rule already covered (references, mutability, iteration) applies just as much to how functions pass values around.

■ Defining and calling a function

DEF, RETURN

```
>>> def make_chai(flavor):
...     return flavor + " Chai"
...
>>> make_chai("Masala")
'Masala Chai'

>>> def greet():           # no return statement
...     print("Welcome to the chai stall")
...
>>> result = greet()
Welcome to the chai stall
>>> result
>>> print(result)
None
```

► EVERY FUNCTION RETURNS SOMETHING

A function with no explicit `return` statement doesn't return "nothing" — it returns `None`, exactly like an empty `.readline()` signaled end-of-file in Chapter 09. `print()` itself always returns `None`, which is why `x = print("hi")` leaves `x` set to `None`.

■ Default arguments

PARAMETERS WITH DEFAULTS

```
>>> def make_chai(flavor="Masala", sugar=True):
...     sweetness = "with sugar" if sugar else "no sugar"
...     return f"{flavor} Chai, {sweetness}"
...
>>> make_chai()
'Masala Chai, with sugar'
>>> make_chai("Ginger")
'Ginger Chai, with sugar'
>>> make_chai("Ginger", False)
'Ginger Chai, no sugar'
>>> make_chai(sugar=False)           # keyword argument – skip flavor by name
'Masala Chai, no sugar'
```

► THE MUTABLE DEFAULT ARGUMENT TRAP

This is one of the most common real bugs in Python, and it follows directly from Chapter 03's reference rules. A default value is evaluated **once**, when the function is defined — not on every call. If that default is a mutable object like a list, every call that relies on the default shares the exact same object:

```
>>> def add_topping(topping, tray=[]):
...     tray.append(topping)
...     return tray
...
>>> add_topping("Ginger")
['Ginger']
>>> add_topping("Mint")           # same list object as before – it remembers!
['Ginger', 'Mint']
```

The fix is the same idiom used throughout this notebook for avoiding shared references: default to `None`, then create a fresh list inside the function body.

```
>>> def add_topping(topping, tray=None):
...     if tray is None:
...         tray = []
...     tray.append(topping)
...     return tray
```

■ *args and **kwargs

Sometimes the number of arguments isn't known in advance. `*args` collects any extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dictionary.

VARIABLE-LENGTH ARGUMENTS

```
>>> def order_summary(*items, **details):
...     print("Items:", items)
...     print("Details:", details)
...
>>> order_summary("Masala Chai", "Samosa", table=4, paid=True)
Items: ('Masala Chai', 'Samosa')
Details: {'table': 4, 'paid': True}
```

`items` arrives as a tuple — Chapter 08's immutable collection — while `details` arrives as a regular dict, with all the `.get()` / `.items()` behavior from that same chapter.

■ Scope — local vs global

LOCAL SHADOWING & GLOBAL

```
>>> counter = 0
>>> def increment():
...     counter = counter + 1  # reading before assigning, inside the function
...     return counter
...
>>> increment()
UnboundLocalError: cannot access local variable 'counter' where it is not associated with
a value

>>> def increment():
...     global counter          # explicitly refer to the outer/global name
...     counter = counter + 1
...     return counter
...
>>> increment()
1
>>> increment()
2
```

► WHY THE FIRST VERSION FAILS

The moment Python sees an assignment to a name anywhere inside a function body (`counter = ...`), it treats that name as **local** to the whole function — even on lines that run before the assignment. The `global` keyword is what tells Python to use the outer name instead of creating a new local one.

■ Functions are objects too

This is the direct payoff of Chapter 03's mental model. A function name is just another name pointing at an object in memory — the same as a list or a number — which means functions can be assigned to variables, stored in data structures, and passed around like any other value.

FIRST-CLASS FUNCTIONS

```
>>> def masala_chai(): return "Masala Chai"
>>> def ginger_chai(): return "Ginger Chai"
>>>
>>> menu = {"masala": masala_chai, "ginger": ginger_chai}  # no () – storing the
function itself
>>> menu["masala"]()  # () is what actually calls it
'Masala Chai'

>>> order = masala_chai  # order now points at the SAME function object
>>> order is masala_chai
True
>>> order()
'Masala Chai'
```

Note the parallel with Chapter 03: `order = masala_chai` doesn't copy the function — it creates a second name pointing at the exact same object, confirmed by `is` rather than just `==`.

■ Lambda — short, anonymous functions

LAMBDA EXPRESSIONS

```
>>> square = lambda x: x ** 2
>>> square(5)
25

>>> teas = ["Earl Grey", "Chai", "Oolong", "Mint"]
>>> sorted(teas, key=lambda t: len(t))  # sort by length, shortest first
['Chai', 'Mint', 'Oolong', 'Earl Grey']
```

A `lambda` is just a function object without a name attached via `def` — everything from the section above about functions being ordinary objects still applies.

■ Closures — functions that remember

A FUNCTION RETURNING A FUNCTION

```
>>> def chai_maker(base_flavor):
...     def add_spice(spice):
...         return f'{base_flavor} Chai with {spice}'
...     return add_spice
...
>>> masala_maker = chai_maker("Masala")
>>> masala_maker("cardamom")
'Masala Chai with cardamom'
>>> masala_maker("cinnamon")
'Masala Chai with cinnamon'
```

► WHAT MAKES THIS A CLOSURE

The inner function `add_spice` keeps access to `base_flavor` from the outer function's scope, even after `chai_maker` has already finished running and returned. Python's garbage collector — introduced in Chapter 03 — doesn't reclaim `base_flavor` because `add_spice` still holds a live reference to it.

CHAPTER 11

Closing Summary & What Comes Next

Ten chapters, one continuous thread: almost everything here traces back to two ideas introduced on page one — Python treats every value as an object in memory, and the PVM executes one instruction at a time. Below is the complete map of what this notebook now covers, plus a natural next-topics list for continued study.

■ The complete map

CHAPTER	CORE IDEA TO CARRY FORWARD
01 — How Python Runs	Source → bytecode → PVM execution loop.
02 — Shell & Modules	The REPL evaluates immediately; modules are reusable, pre-written code.
03 — Mutability & Memory	Variables are names pointing at objects — not boxes holding values.
04 — Numbers	Arbitrary integer precision, float imprecision, and base conversion.
05 — Booleans & Sets	<code>bool</code> is a subtype of <code>int</code> ; sets support algebraic operations.
06 — Strings	Immutable; every method returns a new string.
07 — Lists	Mutable; slicing is a powerful read <i>and</i> write tool.
08 — Dicts & Tuples	Dicts map keys to values; tuples are the immutable counterpart to lists.
09 — Iterators	<code>iter()</code> + <code>next()</code> + <code>StopIteration</code> powers every loop.
10 — Functions	Functions are objects too; scope and closures follow the same reference rules as everything else.

■ Where to go from here

The natural continuation from functions is object-oriented Python — classes are, after all, just another kind of object that bundles functions (methods) together with the data they operate on, governed by the exact same mutability and reference rules built up across this notebook.

CLASSES & OOP

`class`, `__init__`, instance vs class attributes, inheritance, and the dunder methods first seen with `__next__()` in Chapter 09.

EXCEPTION HANDLING

`try` / `except` / `finally` — formalizing the error types already met in Chapters 02, 04, 06, and 08.

DECORATORS

A direct extension of "functions are objects" and closures — a decorator is just a function that wraps another function.

GENERATORS

`yield` builds a custom iterator without manually writing `__next__()` — the natural successor to Chapter 09's iterator protocol.

End of notebook.